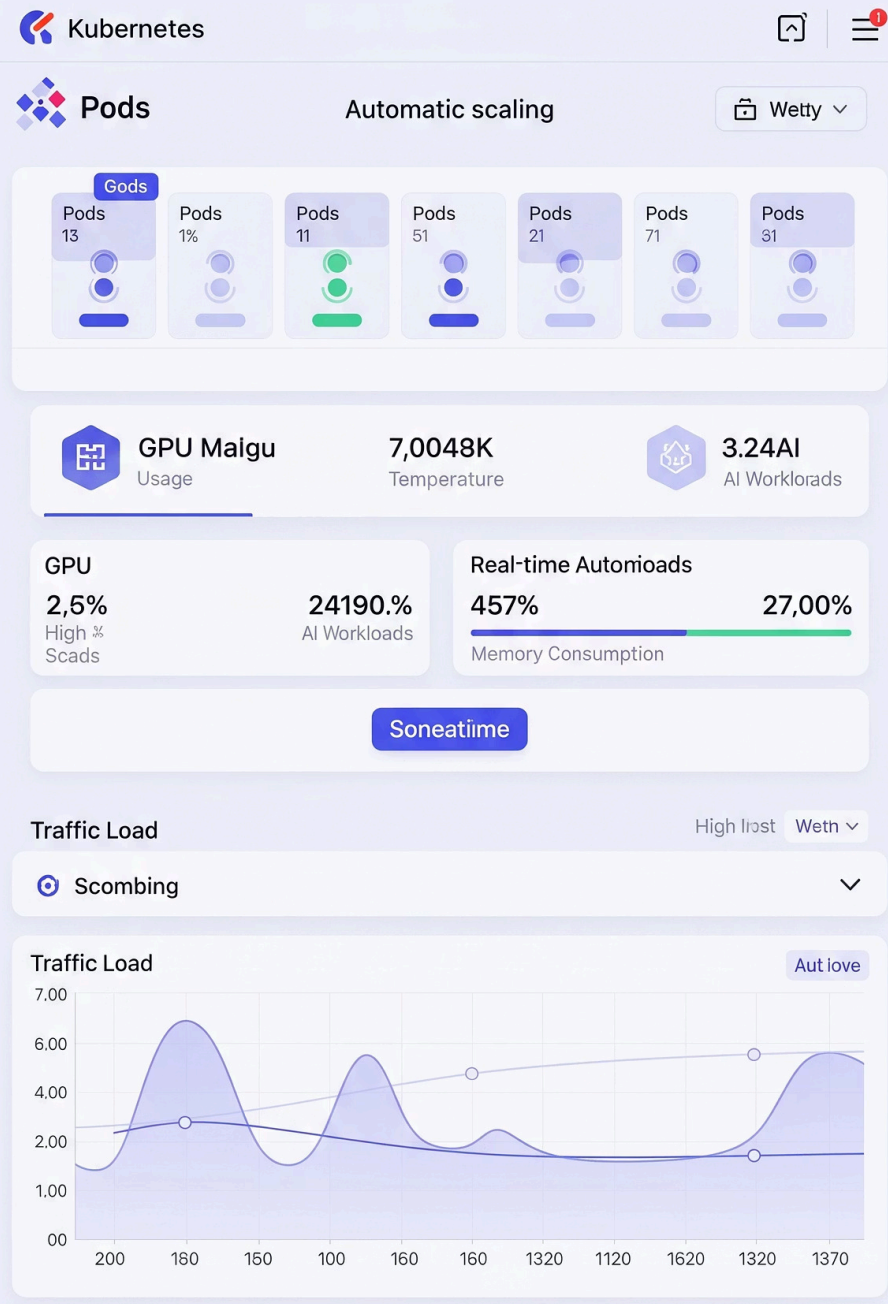


Horizontal Pod Autoscaling for AI Inference

A technical guide for DevOps and ML engineers managing Kubernetes-based AI inference services



Why Autoscaling Matters for AI



Variable Traffic

Inference workloads experience unpredictable spikes and idle periods, creating unique scaling challenges



Cost Optimization

Overprovisioning GPUs and CPUs wastes thousands in cloud spend that could be allocated elsewhere



Performance Impact

Underprovisioning leads to increased latency, failed requests, and degraded user experience

Kubernetes **Horizontal Pod Autoscaler (HPA)** provides the dynamic scaling mechanism needed to balance these competing concerns

What Is Horizontal Pod Autoscaling?



HPA is a Kubernetes resource that:

- Automatically adjusts **replica count** of pods based on observed metrics
- Watches **CPU, memory, or custom metrics** to make scaling decisions
- Maintains workload responsiveness under variable load
- Runs continuously in the background without manual intervention

Ideal for AI inference APIs where user traffic is unpredictable and resource demands fluctuate

How HPA Works



Metrics Collection

Metrics Server continuously collects resource usage data from all pods in the cluster



Threshold Comparison

HPA controller compares actual metrics against target thresholds defined in the HPA specification



Scale Up Decision

If load increases above target, controller calculates new replica count and scales deployment up



Scale Down Decision

If load drops below target, controller waits for stabilization period, then reduces replicas

The HPA controller implements a control loop that runs every 15 seconds by default, continuously balancing replicas to match actual demand

HPA Example (CPU Target)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ai-inference-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ai-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

This HPA example:

- Targets the `ai-api` deployment
- Maintains between **2-10 replicas**
- Scales to keep CPU utilization at **~70%**
- Uses the `autoscaling/v2` API (stable)



HPA for AI Inference Workloads

Handling Inference Spikes

Automatically scales API pods during peak traffic periods, such as business hours or marketing events, when requests to LLMs or vision models spike

Latency Management

Maintains consistent low latency by ensuring enough pods are available to process inference requests without queuing

Cost Efficiency

Reduces expensive GPU/accelerator costs during off-peak hours by scaling down pods when they're not needed

HPA ensures both reliability and cost-effectiveness for **production-grade inference APIs** where performance expectations are high



Custom Metrics for AI

Standard CPU metrics often don't accurately reflect AI workload resource consumption. Custom metrics provide more precise scaling signals:

GPU Utilization

Scale based on actual GPU compute or memory usage

Request Latency

Scale when p95/p99 inference times exceed thresholds

QPS (Queries Per Second)

Scale directly based on traffic volume to the service

Tools

- **Prometheus Adapter** - exposes custom metrics to Kubernetes API
- **KEDA** - supports event-driven scaling
- **DCGM Exporter** - exports NVIDIA GPU metrics

Autoscaling in Cloud AI Services

AWS EKS

Integrates with CloudWatch metrics for custom scaling. Use Container Insights to monitor and scale based on GPU metrics. Supports Prometheus with AWS Managed Prometheus.

GCP GKE

Native support for NVIDIA GPU metrics. GKE Autopilot automatically handles node provisioning. Cloud Monitoring provides AI-specific metrics integration.

Azure AKS

Autoscale with Application Insights metrics. AKS supports KEDA for advanced scaling scenarios. Azure Monitor integration provides GPU and custom metrics.

Cloud-managed AI services (Vertex AI, SageMaker, Azure ML) implement autoscaling by default, with a **scaling-first architecture** that's optimized for variable inference loads

Limitations of HPA



- **Scaling Delay** - Container startup and model loading create cold-start latency (potentially minutes for large models)
- **Not for Training** - Unsuitable for batch training jobs (use Job/TFJob/PyTorchJob instead)
- **Stateless Only** - Works best for stateless inference services
- **GPU Complexity** - GPU fractional allocation and bin-packing are more challenging than CPU
- **Tuning Required** - Requires careful min/max replica configuration based on workload

Best Practices



Redundancy

Always set **minReplicas > 1** to maintain high availability during scaling events or node failures



Readiness Probes

Implement proper readiness probes to ensure pods only receive traffic after models are fully loaded



Monitoring

Create dedicated Prometheus + Grafana dashboards to visualize scaling events and resource usage



Multi-level Scaling

Combine HPA with Cluster Autoscaler to ensure nodes are available when pod count increases

Most critically: **Load test** your autoscaling configuration with realistic traffic patterns before deploying to production to validate behavior

Key Takeaways

1 Dynamic Resource Utilization

HPA automatically scales pods based on demand, crucial for variable AI inference traffic patterns

2 Metric Flexibility

Supports CPU, memory, and custom GPU/latency metrics to accurately capture AI workload needs

3 Business Benefits

Balances cost optimization with performance, saving infrastructure expenses while maintaining responsiveness

Next Steps

- Implement basic CPU-based HPA
- Set up Prometheus for custom metrics
- Create scaling dashboards
- Test with load simulation
- Document scaling policies

Horizontal Pod Autoscaling is a **must-have component** for production-ready AI infrastructure in Kubernetes environments